SIMATS ENGINEERING

ASSESSMENT TOOL 1 – Medium (Scenario Based Assessment)

COURSE CODE: CSA09 **COURSE NAME: Programming in Java**

COURSE OUTCOME COVERED

CO2: *Implement Collection Framework interfaces such as List, ArrayList, Set, Map, HashTable, and HashMap using iterators and generics to store, retrieve, and process groups of objects in web applications.(BL3,BL4)*

QUESTIONS: 5 **Total Marks: 100**

ANNEXURE A

Scenario Based Assessment

Code of Conduct:

I G Vishnu Madhav (192425192) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis				
Application of Concepts				
Analytical & Decision-Making Skills				
Solution Design & Approach				
Clarity, Justification & Presentation				

Scenario Based Assessment

Q1: Student Registration System

Suppose that a university system stores student data, where each student must be unique and can enroll in multiple courses.

(a) Enumerate three Collection interfaces suitable for this system.

(b) Design a Collection structure (diagram/representation) to store students and their enrolled courses.

(c) Write a Java program using HashSet and HashMap with Generics to:

- Add students
- Map students to courses
- Display data using Iterator

Q2: E-Commerce Cart System

Suppose that an online shopping system maintains a cart where products are added, removed, and displayed in order.

(a) List three Collection classes applicable for cart management.

(b) Represent how cart items are stored using a List-based structure.

(c) Write a Java program using ArrayList and Iterator to:

- Add items
- Remove an item
- Display all items

Q3: Library Management System

Suppose that a library stores books with unique ISBN numbers and needs fast retrieval.

(a) Name three Map implementations in Java.

(b) Design a schema-like representation using Map for storing ISBN and book details.

(c) Write a Java program using HashMap to:

- Insert book details
- Retrieve a book
- Display all books

Q4: Employee Data Processing

Suppose that a company processes employee records and filters employees based on salary.

(a) List three advantages of Generics in Collections.

(b) Represent how employee data is stored using List with Generics.

(c) Write a Java program using Iterator to:

- Store employee objects
- Display employees with salary > 50,000

Q5: Online Voting System

Suppose that an online voting system ensures one vote per user and counts votes efficiently.

(a) Identify three suitable Collection types for this system.

(b) Design a structure using Set and Map for voters and vote counting.

(c) Write a Java program using HashSet and HashMap to:

- Record votes
- Prevent duplicate voting
- Display results

Q1: Student Registration System

Description

A university system manages student records where each student must be unique and can enroll in multiple courses. Collections help efficiently store and manage such relationships.

(a) Enumerate three Collection interfaces suitable for this system.

Answer:

1. **Set** – Ensures uniqueness of students
2. **List** – Maintains ordered list of courses
3. **Map** – Maps students to their enrolled courses

(b) Design a Collection structure

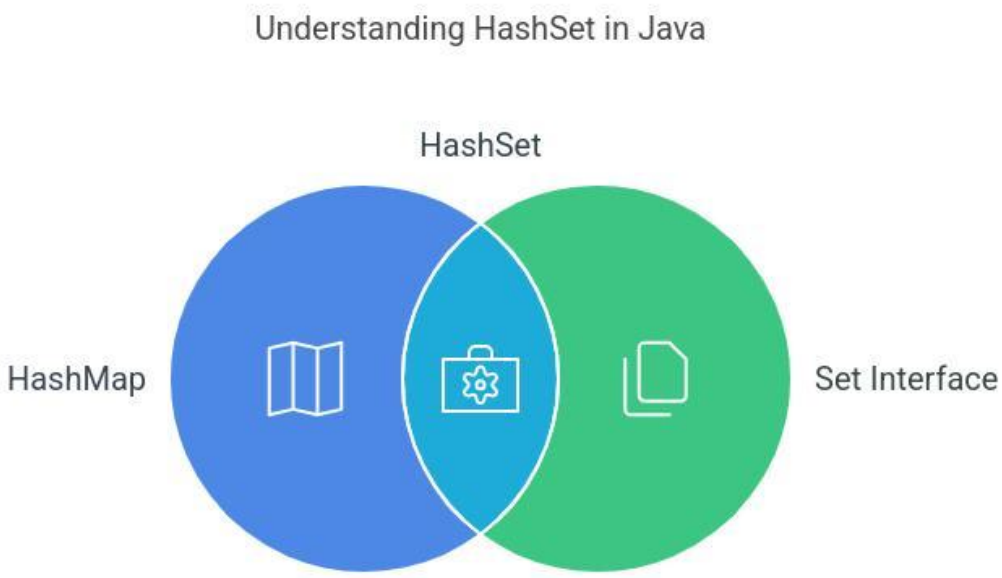
Answer (Representation):

HashSet<Student> students

HashMap<Student, List<String>> courses

Diagram:

Student → [Course1, Course2, Course3]



(c) Java Program

Aim

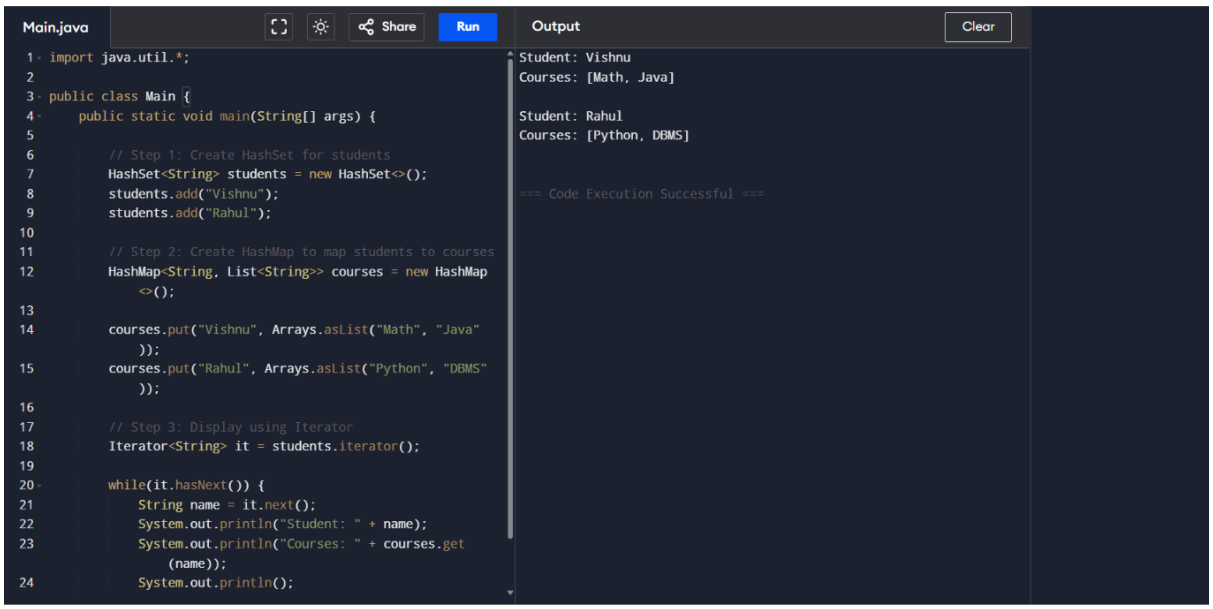
To store unique students and map them to courses using HashSet and HashMap.

Algorithm

1. Create HashSet for students
2. Create HashMap to map students to courses
3. Add students
4. Assign courses
5. Use Iterator to display

Program

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        HashSet<String> students = new HashSet<>();
        students.add("Vishnu");
        students.add("Rahul");

        HashMap<String, List<String>> map = new HashMap<>();
        map.put("Vishnu", Arrays.asList("Math", "Java"));
        map.put("Rahul", Arrays.asList("Python", "DBMS"));

        Iterator<String> it = students.iterator();
        while(it.hasNext()) {
            String s = it.next();
            System.out.println(s + " -> " + map.get(s));
        }
    }
}
```



Q2: E-Commerce Cart System

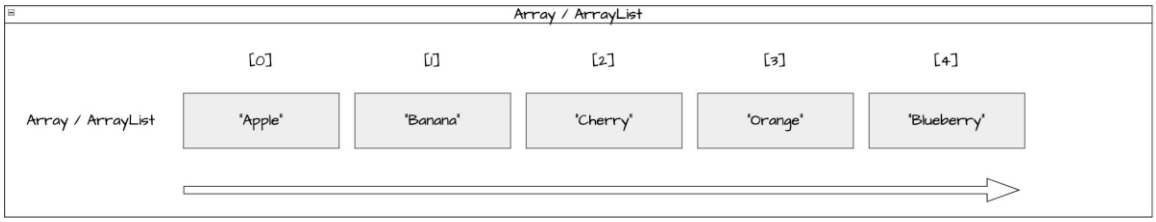
Description

An online shopping system uses a cart to store products dynamically. Items must be added, removed, and displayed in order.

(a) List three Collection classes

Answer:

1. ArrayList
2. LinkedList
3. Vector



(b) Representation

List<String> cart = new ArrayList<>();

(c) Java Program

Aim

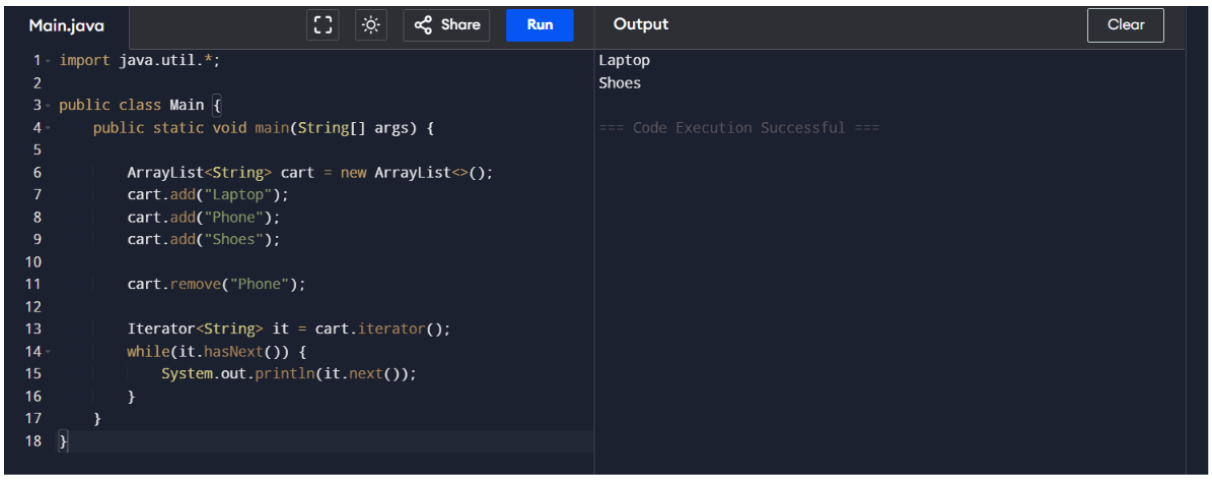
To manage cart items using ArrayList and Iterator.

Algorithm

1. Create ArrayList
2. Add items
3. Remove item
4. Display using Iterator

Program

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        ArrayList<String> cart = new ArrayList<>();
        cart.add("Laptop");
        cart.add("Phone");
        cart.add("Shoes");

        cart.remove("Phone");

        Iterator<String> it = cart.iterator();
        while(it.hasNext()) {
            System.out.println(it.next());
        }
    }
}
```



Q3: Library Management System

Description

A library system stores books using unique ISBN numbers. Fast retrieval is required, making Map suitable.

(a) Three Map implementations

Answer:

1. HashMap
2. TreeMap
3. LinkedHashMap

(b) Representation

HashMap<ISBN, Book>

(c) Java Program

Aim

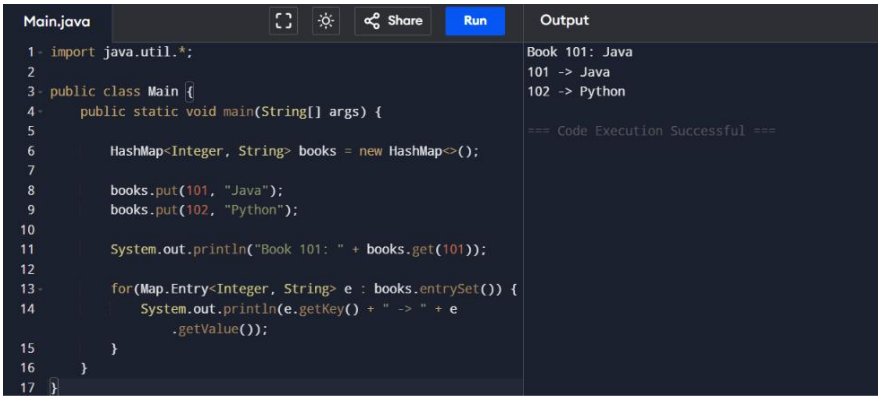
To store and retrieve books using HashMap.

Algorithm

1. Create HashMap
2. Insert books
3. Retrieve using key
4. Display all

Program

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        HashMap<Integer, String> books = new HashMap<>();

        books.put(101, "Java");
        books.put(102, "Python");

        System.out.println("Book 101: " + books.get(101));

        for(Map.Entry<Integer, String> e : books.entrySet()) {
            System.out.println(e.getKey() + " -> " + e.getValue());
        }
    }
```




```
}
```

Q4: Employee Data Processing

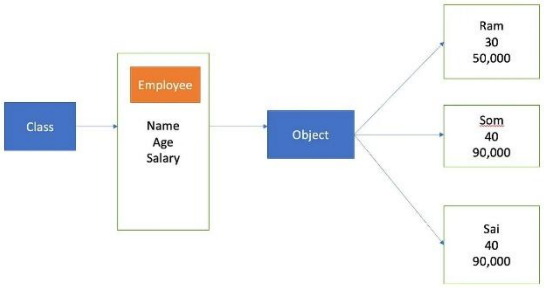
Description

A company processes employee data and filters employees based on salary using Generics and Collections.

(a) Advantages of Generics

Answer:

1. Type safety
2. No type casting required
3. Code reusability



(b) Representation

List<Employee>

(c) Java Program

Aim

To filter employees based on salary using Iterator.

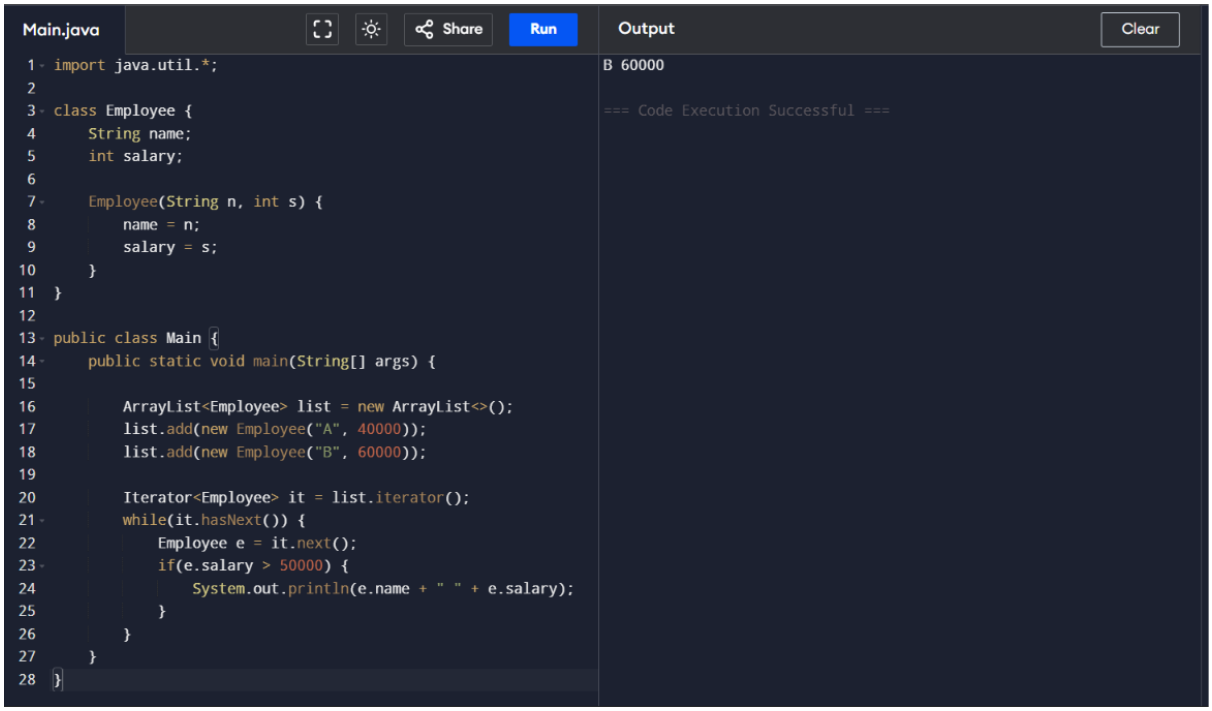
Algorithm

1. Create Employee class
2. Store objects in List
3. Use Iterator
4. Display salary > 50000

Program

```
import java.util.*;

class Employee {
    String name;
    int salary;

    Employee(String n, int s) {
        name = n;
        salary = s;
    }
}

class Main {
    public static void main(String[] args) {
        ArrayList<Employee> list = new ArrayList<>();
```

```
        list.add(new Employee("A", 40000));
        list.add(new Employee("B", 60000));

        Iterator<Employee> it = list.iterator();
        while(it.hasNext()) {
            Employee e = it.next();
            if(e.salary > 50000) {
                System.out.println(e.name + " " + e.salary);
            }
        }
    }
}
```



Q5: Online Voting System

Description

An online voting system ensures each user votes only once and counts votes efficiently.

(a) Suitable Collections

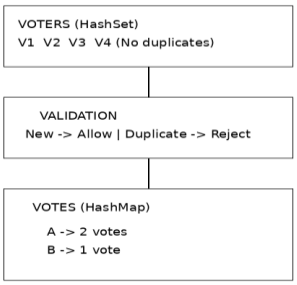
Answer:

1. HashSet – to prevent duplicate voters
2. HashMap – to count votes
3. List – optional for storing candidates

(b) Representation

Set<Voters>

Map<Candidate, Votes>



(c) Java Program

Aim

To record votes and prevent duplicate voting.

Algorithm

1. Create HashSet for voters
2. Create HashMap for votes
3. Add vote if not duplicate
4. Display results

Program

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        HashSet<String> voters = new HashSet<>();
        HashMap<String, Integer> votes = new HashMap<>();

        String voter = "V1";
        String candidate = "A";

        if(voters.add(voter)) {
            votes.put(candidate, votes.getOrDefault(candidate, 0) + 1);
        } else {
            System.out.println("Duplicate Vote!");
        }

        System.out.println(votes);
    }
}
```